Task 1

First token latency

It was extracted from output of
`./build/bin/llama-bench -m ./models/llama32-q4.gguf -p 1 -o json`

"avg_ns": 74033020,

avg_ns/1000000 = 74033020/1000000 = **74.033 ms**

Average token generation speed

It was extracted from output of
`./build/bin/llama-bench -m ./models/llama32-q4.gguf -n 10 -o json`
"n_gen": 10,
"avg_ns": 665234026,
"avg_ts": 15.033192,

There are two methods to calculate the speed:
1/avg_ts = 1/15.033192 = 0.066519472 s = **66.519 ms**
avg_ns / n_gen = 66523403 ns = **66.523 ms**

RAM usage during inference

It was extracted from logs
`./build/bin/llama-cli -m ./models/llama32-q4.gguf -p "What is bitcoin?" -r "<|im_end|>" -r "<|im_start|>" --log-verbose --log-file ./exec.log`

memory breakdown [MiB]	total	free	self	model	context	compute	unaccounted
- Host		14844 =	308 +	14336 +	200		
- CPU_REPACK		1512 =	1512 +	0 +	0		

scripts for gguf conversion and quantization

```
python convert_hf_to_gguf.py ./models/llama32-files/ --outtype f16 --outfile ./models/llama32-full.gguf &> conversion.log

./build/bin/llama-quantize ./models/llama32-full.gguf ./models/llama32-q4.gguf Q4_0 &> quantization.log
```

output logs

https://drive.google.com/file/d/1F2pphKUT92JR6-QBI7xrmhvDiUWkY0xe/view?usp=drive_link
https://drive.google.com/file/d/1-EXL-rHxlBekvaSoh0SEC-Ym87yKRvBe/view?usp=drive_link

Screenshot

See Task1 - Screenshot.png

Task 2

Output quality

From my experience:
Q4_0 often produces no answer or rephrases it.
Q4_HQQ provided slightly different reasonable answers. However, without -n 100 limitation it has a tendency to looping.

First token latency

189.391 ms CPU / **66.17 ms** NVIDIA GeForce RTX 3060

Average token generation speed

According to 2 methods of calculation: **191.367 ms** or **192.272 ms** / **68.656 ms** NVIDIA GeForce RTX 3060

RAM usage during inference

memory breakdown [MiB]	total	free	self	model	context	compute	unaccounted
- Host			16524 =	1988 +	14336 +	200	

Final comparison

Model	Speed	Size	Quality
Q4_0	66 ms	1828 MB	Low
Q4_HQQ	191 ms	1996 MB	Medium

scripts for gguf conversion and quantization

```
./build/bin/llama-quantize ./models/llama32-full.gguf
./models/llama32-q4_hqq.gguf Q4_HQQ &> quantization.log

./build/bin/llama-cli -m ./models/llama32-q4_hqq.gguf -p "What is
bitcoin?" -r "<|im_end|>" -r "<|im_start|>" -n 100

./build/bin/llama-cli -m ./models/llama32-q4_hqq.gguf -p "What is
bitcoin?" -r "<|im_end|>" -r "<|im_start|>" --cache-type-k q4_hqq
--cache-type-v q4_hqq -n 100
```

Task 3

The flag can be used like:

```
./build/bin/llama-cli --list-devices
```

Available devices:
CUDA0: NVIDIA GeForce RTX 3060 (11909 MiB, 11783 MiB free)

```
./build/bin/llama-cli -m model_name -mb CUDA0 …
```

or

```
./build/bin/llama-cli -m model_name --mmproj-backend CUDA0 …
```

Other

Environment:

Task 1
I've already had llama.cpp downloaded and compiled. And even python So the only modification to environment I had to do was python dependencies install

```
pip install -e .

cmake -B build-d -DCMAKE_BUILD_TYPE=Debug
cmake --build build-d --config Debug
```

And these commands were executed before:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

For machine with GPU NVIDIA GeForce RTX 3060

```
cmake -B build -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=86
-DCMAKE_CUDA_HOST_COMPILER=/usr/bin/g++-11
-DCMAKE_CUDA_COMPILER=/usr/local/cuda-12.8/bin/nvcc -DGGML_NATIVE=OFF
-DGGML_CUDA_FATTN=OFF

cmake --build build --config Release -j
```

Issues:

Task 1

Sometimes llama went to infinite looping or produced empty response.

Task 2

It seems that the llama.cpp project was a little bit changed after you composed your task. At least `src/llama-quantize.cpp is now src/llama-quant.cpp`
But anyway chasing Q4_0 was the best advice

quantize_row_q4_0_impl has two ways of working. If it has no quant_weights parameter, it uses quantize_row_q4_0_ref, otherwise it has an algorithm using make_qx_quants function (as well as q3_K, q6_K and q5_0 quantization functions). I skipped the second brunch for q4_hqq and always use quantize_row_q4_hqq_ref in quantize_row_q4_hqq_impl

I spent a lot of time trying to create an AVX2 implementation. Agents helped me a lot, but they are not able to do it directly. And I wasn't really familiar with AVX2. But finally I was happy to see 5 tokens per second instead of one.

While compiling on a machine with CUDA I faced an issue with the 'movmatrix' feature support. It was strange because CUDA 12.8 was used. Anyway I have to switch off FATTN.